In this case, the Enum.sort(files) line returns an empty list [], so the pattern [first_file | _rest] doesn't match. The else statement is evaluated and [] matches this pattern, so the error clause executes and an error tuple is returned.

With statements are useful if your function has many steps that could fail, such as issuing, parsing, and handling an external HTTP request. If you want to gracefully handle any errors, use with.

Let's look at the last control flow structure, cond.

Cond Statements

The last control flow structure at our disposal is the most simple. A cond statement evaluates a number of clauses until the first truthy value is reached. The first matching clause will execute the body statement. Elixir doesn't provide an else if syntax, but cond statements fill this gap.

The other control flow structures are more commonly seen than cond is, but it still has its place. Use a cond statement when checking multiple separate expressions at once. And when your check uses a function (like String.contains?/2), then you have to use a cond statement because you can't use functions inside of pattern matches. Let's take a look:

```
$ iex
iex>
cond do
  false -> "no"

  true ->
    IO.puts "Multiple-lines can be used"
    "yes"
end

# Output:
Multiple-lines can be used
"yes"
```

The power of cond is that you can execute functions in the clauses—something that you cannot do with pattern matching. So you can check any statement:

```
$ iex
iex> str = "The fox jumped over the dog"
iex>
cond do
  String.contains?(str, "z") -> "not this"
  2 * 2 == 100 -> "certainly not this"
  length(String.split(str, " ")) >= 5 -> "5 words or more"
end

"5 words or more"
```